Evaluating GPT-5.4-mini for Gherkin Test Generation: An Empirical Study

Nguyen Minh Hoang, Nguyen Hieu An, and Le Thi Quynh Chi*

FPT University, Vietnam
chiltq6@fe.edu.vn

Abstract. Large language models can draft behavior-driven development (BDD) scenarios, but valid syntax does not guarantee preserved business intent. We evaluate GPT-5.4-mini on reconstructing 261 project-maintained Gherkin files from Sylius, Apache Fineract and Diaspora from user stories reverse-engineered from them, with one same-project example per prompt. We measure parser compliance with behave and skeleton cosine similarity to the reference. Excluding the three prompt exemplars ($N = 258$), 255 files (98.8%, 95% CI [96.6%, 99.8%]) are valid Gherkin, above the 80% threshold ($p < 0.001$). The median similarity is 0.853; a one-sided exact sign test does not show that it exceeds the proposal's 0.85 reference point ($p = 0.209$, $\delta = 0.054$). Only the e-commerce project exceeds it; the decision changes at 0.84. In an exploratory audit of 60 high-similarity files, 25 contain a business-logic defect (AI-drafted, human-reviewed labels; $\kappa = 0.571$ and 0.609 against an independent rater), and seeded defects barely change the score. In a same-time rerun, removing the example lowers the median to 0.811, while another exemplar moves single files in both directions with no detectable average effect. Within this reconstruction setting, GPT-5.4-mini is a drafting aid whose business logic needs human review, and embedding similarity alone cannot certify generated Gherkin.

Keywords: Large language models · Gherkin · Behavior-driven development · Acceptance testing · Software testing · GPT-5.4-mini

1 Introduction

Behavior-driven development expresses expected software behavior in executable or executable-adjacent scenarios, commonly using Gherkin. In practice, the cost of BDD is not limited to writing Given–When–Then syntax. Engineers must preserve domain preconditions, project conventions, data dependencies, and expected outcomes while keeping feature files parsable by the project toolchain. A generated scenario can therefore be syntactically valid yet incomplete as a specification of business behavior.

The three projects in this study illustrate this distinction. A Sylius checkout scenario may depend on channel, country, shipping-method state, and confirmation behavior. Apache Fineract scenarios frequently encode dates, repayment

* Corresponding author.

periods, balances, and data tables. Diaspora scenarios emphasize browser-visible behavior such as navigation, tags, comments, and notifications. Converting a compact Connextra story into project-consistent Gherkin is consequently a constrained generation problem rather than a simple paraphrasing task.

1.1 Research Gap and Motivation

Prior work shows that LLMs can generate BDD artifacts, but existing evaluations differ substantially in data scale, model family, and validation method. Rathnayake et al. evaluated GPT-4, Claude 3, and Gemini on 500 user stories from four proprietary software products and reported a BERTScore F1 of 91.16% for GPT-4 [1]. Karpurapu et al. reported fewer Gherkin-lint syntax errors under few-shot prompting than zero-shot prompting; across their evaluated models, zero-shot outputs accounted for 89% of the observed syntax errors [2]. Ferreira et al. evaluated a GPT-4 Turbo industrial acceptance-testing pipeline that generated Gherkin scenarios and downstream Cypress tests [3], while Fernandes et al. compared several LLMs on a much smaller sample from a single production system [4]. These studies establish feasibility, but they do not jointly answer whether a current compact model can produce both parser-ready and structurally close Gherkin across multiple public software domains. In particular, none of the five studies in Table 1 tests whether its semantic measure reacts to business-logic defects, and none varies the example given in the prompt.

This study addresses that narrower gap by evaluating the pinned gpt-5.4-mini-2026-03-17 snapshot on 261 stories from three unrelated open-source projects, using same-project one-shot prompting and two separate outcome measures. The emphasis is not that GPT-5.4-mini is inherently superior to earlier models. Rather, the contribution is an auditable evaluation design that separates syntactic validity from structural similarity on a public, multi-domain benchmark, and that tests how far the similarity metric can be trusted. Because the stories were reverse-engineered from the reference files and the 0.85 reference point was not calibrated, this revised version adds a leakage measurement, a defect audit of high-scoring outputs, a perturbation test of the metric, a sensitivity analysis over reference points, per-project results, a prompt ablation, a same-time rerun with repeated runs and a consensus round on the doubly labelled files.

1.2 Research Questions and Contributions

RQ1 – Structural skeleton resemblance. When GPT-5.4-mini reconstructs feature files from user stories reverse-engineered from them, how is the skeleton cosine similarity to the reference distributed around the reference point of 0.85 fixed in the proposal?
RQ2 – Syntactic validity. Does GPT-5.4-mini generate feature files that the behave Gherkin parser accepts at a rate above 80%?

The paper makes three contributions: (1) a 261-case, multi-domain benchmark derived from project-maintained reference files in Sylius, Apache Fineract, and Diaspora, with its construction and leakage risk documented; (2) an evaluation

that reports parser compliance separately from structural similarity and checks the similarity metric against defect labels, seeded defects and alternative reference points; and (3) a replication package containing the pinned model identifier, prompts, scripts, API logs, outputs, and all revision analyses.

Section 2 reviews related work. Section 3 describes the dataset, the generation pipeline, the metrics and the analyses, and Section 4 reports the results. Sections 5 and 6 discuss the implications and the threats to validity, and Section 7 concludes. Appendix A gives the prompt and additional results, and Appendix B the perturbation test.

2 Related Work

2.1 LLM-Based Gherkin and BDD Generation

Research on LLM-assisted BDD generation has typically emphasized either semantic resemblance or syntactic correctness. Rathnayake et al. [1] evaluated GPT-4, Claude 3 and Gemini on 500 user stories from four proprietary products of one company and reported strong BERTScore performance. They released the dataset, but all of its stories come from a single organization. Fernandes et al. [4] compared seven LLMs under zero-, one- and few-shot prompting on ten test descriptions from a single production system, scoring outputs with METEOR over five repetitions; their qualitative analysis found business-logic flaws, such as a reversed or omitted outcome, among the most frequent errors. Karpurapu et al. [2] evaluated GPT-3.5, GPT-4, Llama-2-13B, and PaLM-2 and found fewer Gherkin-lint syntax errors with few-shot prompting overall; zero-shot outputs accounted for 89% of the observed syntax errors. This supports evaluating example-based prompting, while noting that their validator was a linter rather than a runtime parser.

Industrial pipeline studies provide a complementary view. Ferreira et al. [3] evaluated GPT-4 Turbo in an industrial web-application workflow; all 50 Cypress test cases from their downstream Test Flow step were valid TypeScript, yet a manual analysis found that only 60% reflected their Gherkin scenarios. Fonseca et al. [5] evaluated the AToMIC framework on 13 issues from the BMW MyBMW application and, by a manual keyword check, found 93.3% of the generated Gherkin scenarios syntactically valid. These results demonstrate that LLMs can be integrated into practical workflows, but single-organization or single-application evaluations make cross-domain generalization difficult to assess.

2.2 Test Generation and Semantic Evaluation

Broader software-test generation research shows that syntactic success and fault-revealing or semantic quality should not be conflated. TestBench provides a benchmark for evaluating class-level test-case generation across LLMs [6]. Chang and Shokrolah Shirazi propose a systematic benchmark for assessing LLM test-case generation over generated program structures [7]. XUAT-Copilot uses multiple

LLM-based agents for automated user-acceptance testing of WeChat Pay [8]. For generated code, EvalPlus found pass@k up to 19.3–28.9% lower once the HumanEval tests were extended 80-fold [9], and CodeJudge shows that token- and embedding-based metrics fail on code that is semantically equivalent to the reference but written differently [10]. Where an executable oracle exists it replaces resemblance, as in LIBRO's criterion that a test fail on the buggy version and pass on the fixed one [11], TiCoder's use of tests to clarify user intent [12], and Meta's mutant-guided ACH [13]. Gherkin generation has no such oracle, because a valid scenario can still state the wrong business rule.

For comparison with a reference, sentence embeddings provide a scalable alternative to manual labeling. Sentence-BERT [16] enables dense representations suitable for cosine similarity. However, concrete names and values in raw Gherkin can dominate textual resemblance without representing domain intent. The present study therefore masks quoted values and numbers and keeps only keyword lines before embedding, and it treats the resulting skeleton similarity as a proxy rather than a definitive measure of business correctness. Section 4.4 tests how well that proxy tracks business-logic defects.

Reporting is a further concern. A review of 74 LLM studies in requirements engineering found that only 16% used public datasets and 39% did not state how prompts were selected [14]. Guidelines for empirical software-engineering studies with LLMs require reporting the model version, configuration and date of use, publishing all prompts and explaining how few-shot examples were selected, and they name sensitivity to in-context examples as a validity threat [15].

2.3 Positioning of This Study

Table 1 compares this study with the five BDD-generation studies above on the criteria that bear on our contributions. The contribution is methodological and empirical rather than a claim of model novelty.

Contribution 1, the benchmark. The other four studies with named sources each draw on one company, system or application, and Ferreira et al. and Fonseca et al. do not release their data; Karpurapu et al. use about 50 stories whose domains are not named. Our 261 reference files come from three public projects in different domains and are released with their construction and a measurement of its leakage.

Contribution 2, the evaluation. Karpurapu et al. check syntax with a linter, and the other studies measure semantics with similarity metrics, ratings or an LLM validator. Fernandes et al. describe business-logic flaws qualitatively, but none of the five studies tests whether its semantic measure reacts to business-logic defects. Our defect audit and perturbation test show that files with high skeleton similarity can still violate their acceptance criteria. Three of the studies compare prompting techniques, but none varies which example is given in the prompt; our ablation shows that removing the example lowers skeleton similarity by more than repeated runs vary, while a different exemplar moves single files in both directions without a detectable average effect. The behave parser answers only whether a generated file is valid Gherkin, and skeleton cosine similarity estimates

Table 1. Comparison with BDD-generation studies, as reported in each paper. The line under each study gives its focus, models and prompting, and main finding.

Study	Data and openness	Syntax oracle	Semantic oracle	Human defect labels	Metric perturb.	Exemplar varied
Rathnayake [1]	500 stories, one company; released	none	BERTScore and other similarity, LLM judge	1–5 quality ratings; no defect labels	no	no
	Focus: best model, prompt and input for scenario generation. *Models:* GPT-4, Claude 3, Gemini; zero-shot, few-shot, chain-of-thought. *Finding:* GPT-4 scored highest on similarity, Claude 3 highest with experts and LLM judges; the best prompt differed by model.					
Karpurapu [2]	about 50 public stories; release not stated	Gherkin-lint	none	none	no	no
	Focus: syntax errors and validation accuracy of generated acceptance tests. *Models:* GPT-3.5, GPT-4, Llama-2-13B, PaLM-2; zero-shot, few-shot. *Finding:* GPT-3.5 and GPT-4 produced error-free tests; few-shot prompts gave fewer syntax errors.					
Ferreira [3]	13 stories, one company; confidential	TypeScript check of Cypress code	manual relevance check; helpfulness ratings	three error types in 20 tests	no	no
	Focus: Gherkin and Cypress generation in a company workflow. *Models:* GPT-4 Turbo; one iteratively engineered prompt per step. *Finding:* users found 95% of the scenarios helpful; 60% of the Cypress tests reflected their scenarios.					
Fernandes [4]	10 of 1,286 cases, one system	none	METEOR	qualitative error types	no	no (fixed format)
	Focus: accuracy and stability on free-form test descriptions. *Models:* seven LLMs, e.g. GPT-4 Turbo, Gemini, DeepSeek R1; zero-, one-, few-shot. *Finding:* Gemini had the highest mean METEOR (0.84), but model differences were not significant.					
Fonseca [5]	13 issues, one app; excerpts only	manual keyword check	LLM validator; practitioner survey	none	no	not stated
	Focus: Gherkin, Page Objects and UI tests from issues and code changes. *Models:* local DeepSeek-R1 for Gherkin, DeepSeek-Coder-V2, Gemma3:1b. *Finding:* 93.3% of the scenarios were syntactically valid and all generated UI tests ran.					
This study	261 files, three public projects; released	behave 1.3.3 parser	none; structural skeleton cosine plus defect audit	criterion-level audit of 60 files; agreement and consensus on 15	8 operators, 623 pairs	yes (C1, C2)
	Focus: RQ1 and RQ2. *Model:* gpt-5.4-mini-2026-03-17; same-project one-shot (C0), zero-shot (C1), other exemplar (C2). *Finding:* 98.8% parse; median similarity 0.853 is not above 0.85; high-similarity files still contain defects.					

closeness to the reference, so the two outcomes are reported and interpreted separately.

Contribution 3, the replication package. It contains the model identifier, prompts, scripts, API logs, generated outputs and all revision analyses, so that every number in this paper can be recomputed.

3 Methodology

Fig. 1 shows how the files pass through generation and analysis, with the denominator of each analysis.

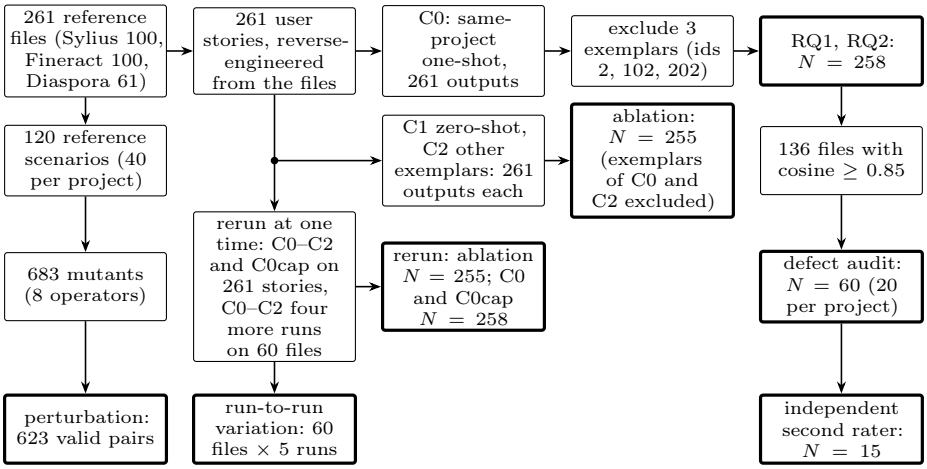


Fig. 1. Data flow and the denominator of each analysis. Bold boxes are analysis sets. Exemplars are excluded so that no prompt example is scored against itself. The rerun repeats the prompts within one time window.

3.1 Dataset Construction

The dataset pairs 261 project-maintained reference files (.feature files) from three open-source projects with 261 user stories. The stories were not collected from a backlog. They were reverse-engineered from the feature files: GPT-5.4-mini (temperature 0) received the text of each feature file and was asked to write one Connextra-style story ("As a [role], I want [feature], so that [benefit]") with as many acceptance criteria as needed to cover all of its scenarios. Only the first 15,000 characters of each file were passed to the model. This truncated 84 of the 261 files, all of them from Apache Fineract (84 of its 100 files, whose stored references have a median length of about 30,000 characters). Each generated file is scored against the whole stored reference, not against that prefix; for 23 Fineract

files the stored reference is itself shorter than the upstream file (see below). Section 6 discusses the leakage risk this construction creates, and Section 4.5 measures it.

Table 2. Dataset composition and sources. The commits are the heads of the clones made on 19 July 2026; .feature counts all feature files at that commit. The three exemplars of the original prompt (one per project) are excluded from the primary analysis.

Project	Domain	Branch, commit	.feature	Selected	Analysed
Sylius [17]	E-commerce	2.3, `9ab28ca`	839	100	99
Fineract [18]	Finance	develop, `5a93304`	147	100	99
Diaspora [19]	Social network	develop, `b7bf1dc`	71	61	60
Total			1,057	261	258

The unit of analysis is a feature file, not a scenario. With the behave 1.3.3 parser, the 244 parseable files among the 258 analysed references contain 2,554 expanded scenarios, and the 255 parseable generated files contain 1,913. The reference files were not screened with a parser before inclusion: 14 of the 261 reference files fail to parse with behave 1.3.3 (13 with behave 1.2.6), 13 of them because their stored text is cut (see below); the full upstream files of these 13 parse. They remain in the dataset, because the metric of Section 3.4 does not require a parse.

Sources and selection (added in revision). The files were collected in two batches. A random sample of 100 files over the three projects (34, 33 and 33) was downloaded on 28 June 2026. No clone of that download was kept, but the Fineract files that match an upstream version date it to the develop branch between 23 and 29 June 2026. To reach 100 files per project, 161 further files were drawn at random from the clones of Table 2; the random seed was not recorded. Diaspora's 71 feature files have only 58 distinct names, because ten names exist in both its desktop and mobile folders; the 61 selected files cover all 58 names. In the first batch the stored reference differs from the upstream file in two ways: the free-text Feature description is missing from 53 Sylius and Diaspora files, which the skeleton drops anyway, and 23 Fineract references end after 30,038 characters of stored text. Scoring against the full upstream files changes only files 114 and 133, both of which stay below 0.85, so the median, the sign test and every RQ1 decision are unchanged (Wilcoxon $p = 0.452$ instead of 0.474). Extraction paths, clone records and the per-file comparison are in the replication package.

The projects were chosen for domain and style diversity. Sylius covers products, carts, channels, shipping and taxes. Fineract contains data-heavy loan workflows with dates, balances, repayment schedules and data tables. Diaspora covers social-network behaviour driven through the browser. The benchmark is a deliberately heterogeneous sample of mature public English-language repositories, not a representative sample of BDD practice.

3.2 Model and Prompt Configuration

All generation requests specified gpt-5.4-mini-2026-03-17 [20] through the OpenAI Chat Completions API [21] with temperature 0; the returned model name was recorded for C1, C2 and the rerun below but not for the original C0. The requests set no maximum-token cap, which deviates from the research proposal's planned cap of 1,500 output tokens. The original condition (C0) is same-project one-shot prompting: each project has its own system message and one example pair (user story and reference Gherkin) from the same project (Listing 1 in Appendix A). The three example files (ids 2, 102 and 202) are also members of the dataset, and their own generated files score 1.000, 0.997 and 1.000 on the metric below. The primary analysis therefore excludes them ($N = 258$); results for all 261 files are reported alongside.

Prompt ablation (added in revision). Two further conditions test how much the results depend on the example. C1 is zero-shot: the same system message and wording, with the example block and the sentence referring to it removed. C2 is one-shot with a different exemplar per project (ids 86, 119 and 203), drawn with a fixed random seed (2026) from the remaining files of each project before any C2 output existed. Model, temperature, system messages and output clean-up are identical in all conditions. The ablation is analysed on the 255 files that are not an exemplar in any condition. Its analysis plan was written before the C1 and C2 runs and is included in the replication package with its SHA-256 hash. It specifies paired Wilcoxon signed-rank tests on per-file cosine and exact McNemar tests on parser outcomes, each against C0 and Holm-corrected. The C1 and C2 runs took place on 16 October 2026. Because the environment used for this analysis could not download the encoder, skeleton cosine for all three conditions was computed with an ONNX export of all-MiniLM-L6-v2 using the same tokenisation, 256-token truncation, mean pooling and normalisation; it reproduces the original C0 scores to within 4×10^{-7}, and a rerun with sentence-transformers (model revision `1110a24`) agrees with it to within 3×10^{-7} in all three conditions.

Same-time rerun (added in revision). The original C0 ran on 19 July 2026, three months before C1 and C2. On 28 October 2026 we therefore ran C0, C1 and C2 again with one script and one logging path, on all 261 stories and four more times on a subset of 60 stories (20 per project, drawn with a fixed seed from the files that are not an exemplar in C0 or C2). The same run included C0cap, C0 with the proposal's cap of 1,500 output tokens, on all 261 stories (Section 6). The 1,764 requests were shuffled with a fixed seed and sent by four parallel workers, so the conditions are interleaved in time. Its analysis plan was committed with its SHA-256 hash before the first request. It repeats the ablation tests against the new C0. On the 60 files it reports, per condition, the within-file standard deviation (SD) of cosine across runs, the intraclass correlation ICC(1,1) and the number of files whose parser outcome or side of 0.85 changes; for C0 and C2 it adds a two-way analysis of variance (file random, prompt fixed), the mean between-prompt effect with a paired t interval over files, and a Welch interval per file. The plan fixes in advance that the original C0 stays the primary result

and that a paired median difference smaller than the pooled within-file SD is not reported as an effect of the prompt.

3.3 Generation and Recording Pipeline

The pipeline loads the stories, builds the prompt for the story's project and calls the API. It removes a Markdown code fence only when the model wraps its output in one; no generated text is edited by hand. Failure handling differed between runs. The C0 notebook set no timeout, so the client library's default of 600 s applied, and retried any failed request, including rate-limit errors, after a 60 s wait with no limit on the number of attempts; it did not check for empty output and logged only successful calls. For C1 and C2 a script made up to five attempts per story after an error, waiting 5, 10, 20, 40 and 60 s; it marked an output that was empty after fence removal, or a story whose five attempts all failed, as not OK with an empty output, to be retried in a later run of the script; such a story would have stayed in the denominator as a parser FAIL and been excluded from the cosine analysis. In both runs the client library could also retry a request twice on its own. None of these events is recorded: all 522 C1 and C2 requests succeeded at the first attempt, no output in any condition was empty, and all 261 C0 requests returned an output. The intervals between consecutive C0 log entries track output length (Pearson $r = 0.995$, at most 61 s), so a 60 s retry wait is unlikely to have occurred. The rerun script set a 600 s timeout and disabled the library's retries, so that every attempt is logged; it made up to five attempts per request, waiting 5, 10, 20 and 40 s, and counted an empty output as a failed attempt; a request still failing after a resumed run would have stayed in the denominator as a parser FAIL. All 1,764 rerun requests succeeded at the first attempt with a non-empty output. Each call is appended to a log with its timestamp and token counts. In the C0 log, the model field is the requested model name written by the script, not the name returned by the API, and the logged cost was computed with outdated prices. Recomputed from the logged token counts (328,257 prompt and 380,494 completion tokens) at the Standard list price of \$0.75 and \$4.50 per million input and output tokens, without cache discounts, the estimated C0 cost is approximately \$1.96 (an estimate from the logged tokens, not an invoice). The C1, C2 and rerun calls log the model name returned by the API; it was the requested snapshot in every call.

3.4 Metric 1: Skeleton Cosine Similarity

Skeleton cosine similarity compares a generated file with its reference after reducing both to a skeleton. The skeleton keeps every line that begins with a Gherkin keyword (Feature, Background, Scenario, Given, When, Then, And, But), including the keyword itself. It removes tags, replaces every double-quoted string with `<PARAM>` and every number with `<NUM>`, and drops all other lines, so data tables, doc strings and Examples tables disappear (Table 7 in Appendix A). Each skeleton is encoded as a whole with all-MiniLM-L6-v2 [16,22], which mean-pools

token embeddings and truncates its input at 256 word-piece tokens. With g and r the embeddings of the generated and reference skeletons,

$$\mathrm{SkelCos}(g, r) = \frac{g \cdot r}{\|g\|\|r\|}. \tag{1}$$

Two properties of this design limit what the metric can show. First, masking makes the score blind to changed numbers and quoted values by construction. Second, truncation means that the score often reflects only the start of a file: 174 of the 258 reference skeletons (67.4%) and 166 of the 258 generated skeletons (64.3%) exceed 256 tokens, including 94 of the 99 Fineract references (median 3,506 tokens). The metric therefore measures the structural resemblance of keyword lines to the reference, not the preservation of business intent; evidence on business intent comes only from the defect audit (Section 3.6). Section 4.4 therefore checks the metric against defect evidence instead of treating it as a measure of business correctness.

The reference point of 0.85 was fixed in the research proposal before the full run. It was not calibrated against human judgement, and it is not an acceptance criterion: no labels independent of the scores were available to calibrate a cut-off. No prior study reports skeleton cosine with this encoder for Gherkin, and the proposal's pilot (30 stories, ids 1–10, 101–110 and 201–210, mean 0.859) used stories that are also part of the evaluated set. We therefore treat 0.85 as an operational reference point and report how the conclusions change between 0.80 and 0.90 (Section 4.3).

3.5 Metric 2: Parser Compliance

Parser compliance is the share of generated files that the behave Gherkin parser accepts. For a generated set S,

$$\mathrm{ESR} = \frac{|\{s \in S \mid \mathrm{parse}(s) = \mathrm{PASS}\}|}{|S|}. \tag{2}$$

We keep the proposal's name, executable syntax rate (ESR), but the check is narrower than the name suggests. Each file is passed to behave's parser (behave 1.3.3 [23]); no step definitions exist, so no step is matched or executed. A PASS means only that the file is valid Gherkin. It does not mean that the scenarios are complete, executable against the application or correct. The submitted version ran behave --dry-run for this check. Without a steps directory, behave stops with a configuration error before parsing the file, and the original script counted that outcome as a PASS. All parser results in this version were therefore recomputed with the parser itself; behave 1.2.6 and 1.3.3 agree on every generated file.

3.6 Defect Audit and Structural Checks (added in revision)

Because neither metric checks business intent, we audited high-scoring outputs for defects. Of the 258 analysed files, 136 have a skeleton cosine of at least

0.85. From these we drew 20 files per project with a fixed seed (2026), giving 60 files. Each file was labelled against its user story and acceptance criteria; the reference was consulted only for project conventions. For each criterion the labeller recorded two facets. The business-logic facet covers wrong or missing numbers, operators, negations and conditions, wrong expected outcomes, untested criteria and contradicting additions. The temporal/causal facet covers missing preconditions, wrong triggering actions, missing or non-causal outcomes and impossible state orderings. Each defect received a severity from 0 (none) to 3 (the test would pass on a system that violates the criterion), and a file counts as defective if it has at least one defect of severity 2 or higher. Line-level evidence was recorded for each defect.

The labels were drafted by an AI coding assistant (OpenAI Codex) and then reviewed by a second author, who retained all of them unchanged. That review was recorded as one confirmation covering all 60 files, not as a decision with reasons per file. To estimate reliability, the first author, who reports not having seen the labels or their aggregate rates before rating, labelled a subset of 15 files fixed in advance (6 Sylius, 3 Fineract, 6 Diaspora) with the same guide. We report Cohen's κ [24] between these independent labels and the reviewed labels. This is agreement between one independent human rater and AI-drafted, human-reviewed labels, not agreement between two independent human raters. Because both values fell below the 0.7 required by our protocol, the reviewing author and the independent rater then discussed the 15 files in the protocol's consensus round and recorded a final label, severity, reason and line evidence for each of the 30 file–facet decisions.

Separately, an automated check parses each file and flags four keyword-order violations per scenario: no Then step, a Given after a When or Then, a Then before any When, and a non-Given step in the Background. These flags encode the checker's conventions and do not by themselves establish a business defect.

Finally, a perturbation test applied eight defect-seeding operators to 120 scenarios extracted from the reference files and compared each mutant with its original using the same metric (Appendix B).

3.7 Statistical Analysis Plan

The confirmatory tests use $\alpha = 0.05$ and follow standard guidance on matching tests to data properties [25]. Effect sizes are reported without categorical labels such as small or large and are interpreted through their magnitude and 95% confidence intervals: an exact interval for δ, derived from the Clopper–Pearson interval of the share of files above 0.85, and percentile bootstrap intervals (10,000 resamples, seed 2026) for η^2_H, Spearman's ρ, Cohen's κ (resampling files) and, outside the rerun plan, the median paired differences of the ablation.

RQ1: exact sign test, with the Wilcoxon test as sensitivity analysis. H_0: the median skeleton cosine is at most 0.85; H_1: it exceeds 0.85. The proposal phrased these hypotheses in terms of the mean and pre-registered a one-sided one-sample Wilcoxon signed-rank test on $x_i - 0.85$. That test addresses the median only if the differences are symmetric, which they are not (Section 4).

Following review, the median is therefore tested with the exact one-sided sign test, a binomial test of the number of files above against the number below 0.85 with ties dropped (none occurred) [26], and estimated with a 95% percentile bootstrap interval (10,000 resamples, seed 2026) [27]. The Wilcoxon test is reported as a sensitivity analysis, and symmetry is checked with the skewness of $x_i - 0.85$ and its bootstrap interval. As effect size we report $\delta = P(x > 0.85) - P(x < 0.85)$, the sign-test statistic expressed as a proportion and a one-sample analogue of Cliff's δ against the reference point. Quartiles use linear interpolation and the SD uses $n - 1$. A non-significant result is read as insufficient evidence for performance above the reference point, not as evidence of poor performance.

RQ2: exact binomial test. Each file has a binary parser outcome. We test H_0: $p_{\mathrm{ESR}} \leq 0.80$ against H_1: $p_{\mathrm{ESR}} > 0.80$ [26] and report a 95% Clopper–Pearson interval [28] and the risk difference from 0.80.

Analyses added in revision. The following analyses were planned after the reviews and are exploratory. Per-project differences are tested with a Kruskal–Wallis test [29], reported with $\eta^2_H = (H - k + 1)/(n - k)$ and followed by pairwise two-sided Mann–Whitney tests with Holm correction [30]. Reference-point sensitivity repeats the RQ1 test for reference points from 0.80 to 0.90. Lexical leakage is measured as the share of distinct word trigrams of the reference file that also occur in the user story, and is related to skeleton cosine with Spearman's ρ.

3.8 Reproducibility and Denominators

The replication package contains the dataset, prompts, scripts, generated outputs, API logs and all revision analyses at https://github.com/simonhoang611/RT-SWT-004-Group4. The primary analysis uses $N = 258$ files, and no generated file was excluded because of its content; only the three exemplars are left out of the primary analysis.

4 Results

4.1 RQ1: Structural Skeleton Resemblance

RQ1 describes the distribution of skeleton cosine similarity of C0 around the reference point of 0.85 and tests whether its median exceeds that point. On the 258 non-exemplar files the median is 0.853 (95% bootstrap CI 0.844–0.859), 0.003 above the reference point. 136 files lie above 0.85 and 122 below, and the exact sign test gives one-sided $p = 0.209$ with $\delta = 0.054$ (95% CI -0.072 to 0.179) (Table 3; Fig. 2 in Appendix A). The differences are not symmetric: they are skewed to the left (skewness -0.41, 95% bootstrap CI -0.76 to -0.08), the lower half of the interquartile range is wider than the upper (0.048 against 0.037), and the mean (0.848) lies below the median. The pre-registered Wilcoxon test therefore addresses the pseudo-median (Hodges–Lehmann estimate 0.850) rather than the median; it gives $p = 0.474$. Including the three exemplars gives $p = 0.161$ (sign test) and $p = 0.355$ (Wilcoxon). Decision: fail to reject H_0 in all four analyses.

Table 3. Skeleton cosine similarity of condition C0. The primary analysis excludes the three prompt exemplars.

Statistic	$N = 258$ (primary)	$N = 261$
Minimum	0.585	0.585
Q1	0.805	0.806
Median	0.853	0.854
Q3	0.890	0.894
Maximum	0.994	1.000
Mean ± SD	0.848 ± 0.068	0.850 ± 0.069
Median 95% bootstrap CI	[0.844, 0.859]	[0.845, 0.861]
Files ≥ 0.85	136 (52.7%)	139 (53.3%)
Sign test, one-sided p	0.209	0.161
Wilcoxon W, p (sensitivity)	16,784, 0.474	17,548, 0.355
Skewness [95% CI]	−0.41 [−0.76, −0.08]	−0.33 [−0.68, −0.01]
δ [95% CI]	0.054 [−0.072, 0.179]	0.065 [−0.060, 0.189]

4.2 Results per Project

The pooled median hides differences between projects (Table 4; Fig. 3 in Appendix A). Sylius is the only project above the reference point: its median is 0.887 and 77.8% of its files score at least 0.85. Fineract (median 0.826) and Diaspora (median 0.831) are significantly below 0.85 by the Wilcoxon test (one-sided $p < 0.001$ and $p = 0.008$); with the sign test, Fineract remains significantly below ($p = 0.001$) but Diaspora does not ($p = 0.12$; 25 files above, 35 below). The projects differ (Kruskal–Wallis $H = 60.87$, df $= 2$, $p < 0.001$, $\eta^2_H = 0.231$, 95% CI 0.144–0.336). Sylius differs from Fineract and from Diaspora (Holm-adjusted $p < 0.001$ for both), while the difference between Fineract and Diaspora is not significant ($p = 0.74$). Spread is also uneven: the interquartile range is largest for Diaspora (0.087) and smallest for Fineract (0.062).

Truncated story input (added in revision). The 84 Fineract files whose stories saw only the first 15,000 characters of the reference do not score lower than the 15 whose stories saw all of it (median 0.827 vs. 0.824; 27 of 84 and 7 of 15 reach 0.85; two-sided Mann–Whitney $p = 0.99$). Rescoring the 84 files against the skeleton of that 15,000-character prefix changes one score, by −0.004, because the encoder reads only 256 tokens and 82 of the 84 prefix skeletons are longer; the Fineract and pooled medians stay at 0.826 and 0.853. With only 15 non-truncated files this comparison is descriptive.

4.3 Sensitivity to the Reference Point

Because 0.85 was not calibrated, we repeated the RQ1 test for reference points between 0.80 and 0.90 (Table 8 in Appendix A). The pooled median lies significantly above every reference point up to 0.84 (at 0.84, sign test $p = 0.002$, Wilcoxon $p = 0.0097$) and above none from 0.85 upwards. The conclusion of RQ1 therefore depends on where the reference point is placed within a few hundredths.

Table 4. Skeleton cosine per project (C0, $N = 258$). The p-values are one-sided Wilcoxon tests against 0.85 in each direction.

Project	N	Median (IQR)	≥ 0.85 [95% CI]	$p_{>0.85}$	$p_{<0.85}$
Sylius	99	0.887 (0.076)	77.8% [68.3, 85.5]	< 0.001	1.000
Fineract	99	0.826 (0.062)	34.3% [25.1, 44.6]	1.000	< 0.001
Diaspora	60	0.831 (0.087)	41.7% [29.1, 55.1]	0.992	0.008
All	258	0.853 (0.085)	52.7% [46.4, 58.9]	0.474	0.526

The project ranking does not: at every reference point Sylius has the largest share of files above it, and Fineract the smallest from 0.83 upwards.

We could not calibrate the reference point against defect labels. The defect audit (Section 4.4) sampled only files at or above 0.85, so it cannot estimate how often files below a given reference point are defective. Within the audited range, cosine did not differ significantly between defective and non-defective files (Mann–Whitney $p = 0.21$; AUC 0.40 when a higher cosine is read as predicting a defect, i.e. defective files tended to score slightly lower). With 60 files from a restricted range, this neither shows nor rules out that the metric discriminates. Defects occurred in 11 of 24 files between 0.85 and 0.875, 10 of 16 between 0.875 and 0.90, 2 of 11 between 0.90 and 0.95, and 2 of 9 at 0.95 or above.

4.4 Defects in High-Similarity Outputs

Defect audit. The audit is an exploratory observation and is not used to validate any conclusion, because its labels did not reach the agreement our protocol required (below). Of the 60 audited files, all with skeleton cosine of at least 0.85, 25 (41.7%, 95% CI [29.1, 55.1]) contain at least one defect of severity 2 or 3: 6 of 20 Sylius files, 16 of 20 Fineract files and 3 of 20 Diaspora files. The two facets flag exactly the same 25 files, so we report them as one result, not as two independent findings. The most frequent business-logic defects are wrong expected outcomes (20 files), missing or wrong conditions (9) and wrong numbers (8); no wrong operator or lost negation was found. On the temporal/causal facet, outcomes without a verifying Then are most frequent (19 files), followed by Then steps that do not follow from the When (9) and missing preconditions (8). The share of 41.7% describes a sample balanced across projects and restricted to high-scoring files; it is not an estimate of the defect rate of the whole corpus.

For example, a criterion in the story of Sylius file 76 expects a cart total of \$73.80 after "one taxed product" is removed from a cart holding three units of a \$100 product and two units of a \$30 product, both taxed at 23%. That total corresponds to removing the \$100 product entirely. The generated scenario removes a single unit and still expects \$73.80, although its own data imply \$319.80. The file scores 0.914, and both the reviewed labels and the independent rater mark it as defective.

Agreement. On the 15 doubly labelled files, the independent rater agreed with the reviewed labels on 12 files for each facet. Cohen's κ was 0.571 (95% CI

0.05–1.00) for the business-logic facet and 0.609 (0.24–1.00) for the temporal/-causal facet. Both values are below the threshold of 0.7 set in our protocol. Of the six disagreements across the two facets, five concern three Diaspora files that the independent rater marked as defective and the reviewed labels did not; the sixth is a Sylius file in the opposite direction. All three Fineract files received the same labels. In the consensus round (28–29 October 2026), the two raters kept the shared label in all 24 agreed decisions and adopted the independent rater's label in all six disagreements, so the three Diaspora files became defective and the Sylius file lost its business-logic defect. After consensus, 6 of the 15 files have a business-logic defect and 8 a temporal/causal defect, against 5 and 5 in the reviewed labels. The other 45 files were not relabelled, so all audit figures above use the reviewed labels; with the consensus labels for the 15 files, 28 of the 60 files would be defective (26 on the business-logic facet).

Structural checks. The keyword-order check flags 5 of the 255 parseable generated files and 40 of the 244 parseable reference files. Generated files thus follow the Given–When–Then order more consistently than the references.

Perturbation test. In the 623 valid scenario pairs, no seeded defect lowered skeleton cosine below 0.8995. Changed numbers and quoted values leave the skeleton identical by construction, and swapped or reordered steps score at least 0.954, although the keyword-order check flags all of them (Appendix B). These results show insensitivity at the scenario level and do not estimate a false-negative rate for whole files.

4.5 Lexical Leakage from Reverse Engineering

The user stories are short compared with the references: a median of 4.9% of the distinct word trigrams of a reference file also occur in its story (Sylius 8.3%, Fineract 1.6%, Diaspora 5.8%). Higher overlap goes with higher similarity in two of the three projects: Spearman's ρ between overlap and skeleton cosine is 0.29 for Sylius (95% CI 0.11–0.46, $p = 0.003$), 0.33 for Diaspora (0.08–0.55, $p = 0.009$) and 0.11 for Fineract (-0.11 to 0.31, $p = 0.30$). The pooled value ($\rho = 0.43$, 0.33–0.53) is inflated by the project effect, since Sylius is high on both measures.

4.6 RQ2: Parser Compliance

RQ2 asks whether more than 80% of the generated files are valid Gherkin. Of the 258 non-exemplar files, 255 pass the parser (98.84%, 95% CI [96.64%, 99.76%], $p < 0.001$; Table 9 in Appendix A). The risk difference from the 80% reference rate is +18.84 percentage points (95% CI [16.64, 19.76], the Clopper–Pearson interval shifted by 0.80). The three failures (files 136, 148 and 154) are an Examples block outside a Scenario Outline in two files and a malformed data table in one. Decision: reject H_0. This corrects the 100% reported in the submitted version, which came from the faulty dry-run check described in Section 3; the decision is unchanged.

4.7 Prompt Ablation

The comparisons below use the same-time rerun (Table 5; 255 common files). Removing the example (C1) lowered skeleton cosine in 193 of the 255 files (median paired difference −0.030, 95% bootstrap CI −0.038 to −0.021; paired Wilcoxon, Holm-adjusted $p < 0.001$). The median fell from 0.852 to 0.811 and the share of files at or above 0.85 from 52.5% to 33.7%. The drop was largest in Fineract, where only 4 of 98 files reached 0.85. The difference is larger than the within-file SD across runs (0.020 for C0, 0.022 for C1; Section 4.8).

Table 5. Prompt ablation on the 255 files that are not an exemplar in any condition, in the same-time rerun (28 October 2026) and in the original runs (C0 on 19 July, C1 and C2 on 16 October). Skeleton cosine medians overall and per project (n = 98, 98, 59), share of files ≥ 0.85 with 95% CI, and parser PASS count.

Condition	Median	≥ 0.85 [95% CI]	Sylius	Fineract	Diaspora	PASS
Same-time rerun						
C0 (exemplars 2, 102, 202)	0.852	52.5% [46.2, 58.8]	0.892	0.829	0.829	255
C1 zero-shot	0.811	33.7% [27.9, 39.9]	0.872	0.779	0.830	252
C2 (exemplars 86, 119, 203)	0.873	63.5% [57.3, 69.4]	0.891	0.873	0.841	254
Original runs						
C0	0.853	52.9% [46.6, 59.2]	0.886	0.827	0.835	252
C1	0.819	34.5% [28.7, 40.7]	0.873	0.769	0.825	252
C2	0.875	64.3% [58.1, 70.2]	0.892	0.876	0.840	254

Replacing the exemplars (C2) raised cosine in 140 files and lowered it in 115 (median paired difference +0.008, 95% CI −0.001 to 0.015; Holm-adjusted $p < 0.001$; the Wilcoxon test concerns the pseudo-median of the differences, the interval their median). Under C2 the median is 0.873 and 63.5% of the files reach 0.85, mostly because of Fineract (median 0.829 under C0, 0.873 under C2), while Sylius and Diaspora change by less than 0.015. The paired difference is smaller than the within-file SD across runs (0.020 for C0, 0.019 for C2), so by the rule fixed in the plan we do not report it as an effect of the exemplars.

Parser compliance was similar under all three prompts. Under C0, C1 and C2, 255, 252 and 254 of the 255 files parse (exact McNemar against C0, Holm-adjusted $p = 0.50$ and 1.0); the tests do not establish equivalence. Outputs were longer under C1 and C2 than under C0 (median 750 and 752 completion tokens against 498). The original ablation, with C0 run three months before C1 and C2, gave the same direction and similar paired differences (−0.028 and +0.008; Table 5).

4.8 Run-to-Run Variation (added in revision)

Repeating a prompt rarely reproduced its output: on the 60 repeated files the five outputs were identical for one file under C0 and for none under C1 or C2. The scores are more stable. ICC(1,1) is 0.921 (95% CI 0.888–0.948) for C0, 0.904 (0.864–0.936) for C1 and 0.929 (0.898–0.953) for C2, and the pooled within-file SD is 0.020, 0.022 and 0.019. The parser outcome changed between runs for 0, 2 and 2 of the 60 files, but 19 (32%), 13 (22%) and 14 (23%) files lay on both sides

of 0.85 in different runs. The new and the original C0 differ in the same way: 2 of 258 outputs are identical, the median per-file difference is 0.000 (IQR −0.018 to 0.014), and 45 files (17%) lie on different sides of 0.85. The decisions do not change: the new C0 has median 0.852 (95% CI 0.843–0.862) with 135 files above 0.85 and 123 below (sign test $p = 0.247$), and 258 of 258 files parse.

For C0 and C2, the within-prompt SD is 0.020, the file-by-prompt SD 0.042 and the between-file SD 0.055. The mean between-prompt effect (C2 minus C0) is +0.004 (95% CI −0.011 to 0.020). Per file, the Welch 95% interval of the difference lies wholly above 0 for 19 files, wholly below 0 for 17 and includes 0 for 24 (unadjusted for multiplicity). Changing the exemplars thus moves single files by more than repeating the prompt does, but in both directions, so on average it has no detectable effect; Fineract shows the largest mean shift (+0.013; 9 intervals above 0, 6 below).

4.9 Consolidated Answers

RQ1: the median similarity (0.853) lies at the 0.85 reference point and is not demonstrably above it; only Sylius exceeds it, and the test result changes with the reference point within a few hundredths, while repeated runs of the same prompt move a fifth to a third of the repeated files across 0.85. RQ2: yes. Nearly all generated files are valid Gherkin, with or without an example in the prompt.

5 Discussion

5.1 Why Parser Compliance and Structural Similarity Diverge

Nearly all generated files are valid Gherkin (255 of 258), so the model reliably follows the surface grammar. Same-project examples may contribute by showing keyword order, scenario structure, tables and local conventions, which is consistent with prior work showing that examples reduce syntax errors [2] and with industrial studies reporting high syntactic correctness of generated Gherkin [5] and of generated test code [3]. The prompt ablation (Section 4.7) shows that this result does not require the example: in the same-time rerun 252 zero-shot outputs parse against 255 one-shot outputs of 255, and alternative exemplars give 254. The McNemar tests found no difference, although they cannot establish equivalence.

Matching the reference structure is harder, and the per-project results locate the difficulty. Fineract files are long, data-driven and full of dates, rates and balances, and 94 of its 99 analysed references are cut by the encoder's 256-token limit. The truncated story input of 84 Fineract files does not explain the gap, because these files score no lower than the other Fineract files (Section 4.2); its effect on the content of the stories was not measured. Diaspora files depend on browser-level steps and selectors that a story does not state. Sylius checkout and cart scenarios are shorter and closer to their stories. The experiment does not isolate the causal contribution of the remaining factors, but it shows that a single pooled median, which averages one project above the reference point with two below it, is not a meaningful summary of capability across these projects.

5.2 What the Similarity Metric Can and Cannot Show

The ablation and the repeated runs also show how fragile the RQ1 decision is. So many files lie near 0.85 that repeating the same prompt moves a fifth to a third of the 60 repeated files across it. A different exemplar per project raises the median from 0.852 to 0.873, but its paired effect stays within this run-to-run variation, while removing the example lowers the score by more than it. Most of the exemplar-related change occurs in Fineract. We conjecture that this follows from truncation: the encoder sees only the first 256 tokens of 94 of the 99 Fineract references, so the opening of a file, that is its Background and first steps (the skeleton drops all table rows), largely determines the score, and an exemplar that shows a typical opening can raise the score without any evidence that the business content improved. The ablation does not test this explanation.

The revision analyses show that skeleton cosine is a measure of structural resemblance, not of correct behaviour. In the exploratory audit of the high-similarity range, 25 of 60 files contain a defect that changes what the test checks, and in this restricted sample cosine did not rank defective files significantly differently from non-defective ones. Nor does correct keyword order imply correct content. Seeded defects at the scenario level, including dropped Then steps and reordered steps, leave the score at or above 0.8995, and changed numbers or values cannot change it at all because the skeleton masks them. A threshold on this metric, whether 0.85 or any value between 0.80 and 0.90, therefore cannot certify that a generated file preserves its acceptance criteria. We report 0.85 as a descriptive reference point for structural similarity, together with its sensitivity analysis, and do not interpret files above it as correct. The overlap correlations in two projects (Section 4.5) suggest that part of the similarity may also come from wording that the reverse-engineered story carries over from its reference; they do not establish a causal effect.

5.3 Practical Use in BDD Workflows

For stories of this kind, the evidence supports a bounded workflow: use GPT-5.4-mini to draft valid Gherkin, then have a reviewer check expected outcomes, conditions, data values and the coverage of each acceptance criterion. The audit suggests where review effort is needed most: wrong expected outcomes and missing verifying Then steps were the most frequent defects, and defective files were far more frequent in the data-heavy finance domain (16 of 20) than in the other two (6 and 3 of 20). The experiment does not justify promoting generated scenarios into CI/CD as authoritative acceptance tests. A parser pass establishes only that a file is valid Gherkin, and a high similarity score does not establish correctness.

5.4 Relationship to Prior Work

The findings refine, rather than contradict, earlier BDD-generation studies. High parser compliance aligns with the syntactic success reported in industrial pipelines,

for generated Gherkin [5] and for generated Cypress code [3], and with the lower syntax-error counts under few-shot prompting reported by Karpurapu et al. [2]. The similarity result is more cautious than studies reporting strong semantic or text-similarity results [1,4], partly because this experiment uses a different metric, a fixed reference point, a multi-domain sample and a hypothesis test against it. Our defect audit also suggests that high embedding similarity alone, in any of these studies, says little about whether acceptance criteria are preserved. Cross-paper metric values are therefore contextual comparisons, not model rankings.

6 Threats to Validity

For each threat we state the mitigation we actually performed and the risk that remains. Threats that we could only measure or disclose are collected at the end of the section.

6.1 Internal Validity

Reverse-engineered stories. The user stories were written by GPT-5.4-mini from the reference files, so the input can carry wording, structure and scenario order from the very file it is compared with, and the model that wrote the stories is also the model under test. *Mitigation:* we measured story–reference trigram overlap (median 4.9%) and its correlation with similarity (Section 4.5), and we checked the 84 Fineract stories that were written from only the first 15,000 characters of a longer reference; they score no lower than the other Fineract files (Section 4.2). *Residual risk:* overlap correlates with similarity in two of the three projects, so part of the measured similarity likely comes from this construction; structural leakage such as the order and number of criteria is not measured; and no independently written backlog was available, so the results may overstate performance on natural requirements.

Exemplars and pilot. *Mitigation:* the three original exemplars were part of the dataset and are excluded from the primary analysis; both denominators are reported. *Residual risk:* the 30 pilot stories used when the reference point was set are part of the evaluated set.

Prompt ablation. The original ablation ran three months after C0. *Mitigation:* the same-time rerun repeats C0, C1 and C2 with interleaved requests, and its C0 has the same median as the original C0. *Residual risk:* C2 uses one random draw of exemplars, and the rerun cannot show what changed in the hosted service between the two dates.

Hosted model. The serving infrastructure is outside our control. *Mitigation:* we pinned the snapshot, used temperature 0, logged each call, recorded the returned model name for C1, C2 and the rerun, and measured run-to-run variation on 60 files (Section 4.8). *Residual risk:* nondeterminism remains, the original C0 log records only the requested name, and the API returned no system fingerprint, so changes of the serving configuration cannot be detected.

Output cap. The proposal planned a cap of 1,500 output tokens that the original runs did not apply. *Mitigation:* the rerun included this capped condition (C0cap). On the 258 non-exemplar files, 81 outputs stopped at the cap, and 239 files passed the parser (92.6%, 95% CI [88.7%, 95.5%]) against 258 for the rerun C0 (exact McNemar $p < 0.001$); all 19 failures are capped outputs. The median cosine was 0.854, with a median paired difference of +0.0005 (Wilcoxon $p = 0.108$). *Residual risk:* the RQ2 result applies to uncapped generation; under the cap, long files are cut and parser compliance is lower. These two tests are descriptive and uncorrected.

6.2 Construct Validity

Similarity metric. Skeleton cosine is an imperfect proxy for business correctness. *Mitigation:* we checked it against defect labels, seeded defects, reference points from 0.80 to 0.90 and per project. *Residual risk:* it is blind to numbers and quoted values by construction, ignores data tables and Examples, and for about two thirds of the files it sees only the first 256 tokens of a skeleton. Within the audited high-similarity files it did not separate defective from non-defective outputs significantly, scenario-level seeded defects barely changed it, and a valid alternative scenario can score low because it differs from the single project-maintained reference.

Defect labels. The 60 audit labels were drafted by an AI assistant and retained unchanged by a reviewing author, whose review was recorded as one confirmation rather than per-file decisions. *Mitigation:* an independent human rater labelled 15 files (independence rests on the rater's own declaration), and we report κ with 95% intervals. Because κ was below our protocol's threshold, the reviewing author and the independent rater then held the required consensus round on these 15 files (Section 4.4). *Residual risk:* $\kappa = 0.571$ (business logic) and 0.609 (temporal/causal) are below the 0.7 our protocol required, and with 15 files they are imprecise (the intervals reach down to 0.05 and 0.24). The protocol also required relabelling, but no rater who had not seen the labels was available for the other 45 files, so we keep their reviewed labels, report κ as measured before consensus and treat the audit as an exploratory observation rather than as validation. The consensus resolved every disagreement in favour of the independent rater and added defects in three Diaspora files, so the reviewed labels of the other 45 files may undercount defects, in particular in the social-network domain. The two label facets are not independent measurements, and the audit covers only files at or above 0.85.

Perturbation test. *Mitigation:* pairs in which a text does not parse are excluded, and results are reported by operator (Appendix B). *Residual risk:* the test mutates single extracted scenarios, omitted the Background of 74 of 120 source scenarios, and draws several mutants from the same scenario, so its pairs are not independent and do not estimate whole-file sensitivity.

Parser compliance. *Mitigation:* the submitted version's dry-run check, which counted a configuration error as a PASS, was replaced by behave's parser, and all parser results were recomputed. *Residual risk:* a parser pass shows only

that a file is valid Gherkin, not successful execution against the application or correct step implementations.

6.3 External and Conclusion Validity

Projects. *Mitigation:* all main results are reported per project. *Residual risk:* the dataset spans three mature, English-language public projects and is not representative of BDD practice; results may differ for short tickets, mixed-language requirements, private conventions or projects with unstable step definitions. The projects contribute unequal numbers of files (99, 99 and 60 after exclusion) and differ significantly in similarity, so pooled results do not generalise across domains.

Thresholds and tests. *Mitigation:* we report how the conclusions change between reference points of 0.80 and 0.90, and the ablation and rerun plans were fixed with their hashes before the runs. *Residual risk:* neither the 0.85 reference point nor the 80% threshold is an industry standard, the reference point was not calibrated, and the RQ1 test result changes between 0.84 and 0.85. All analyses added in revision were planned after the reviews and are exploratory; their p-values are not corrected across analyses.

Uncontrolled limitations. The measures above do not control every threat. We had no independently written requirements, could not detect changes in the hosted service, did not execute the generated files against step definitions, and could not relabel the 45 audit files outside the consensus round. The conclusions therefore hold only for reverse-engineered stories from these three projects, in the time window of the runs, and with the similarity metric read as a proxy.

7 Conclusion

This study evaluated how well GPT-5.4-mini reconstructs 261 feature files of three open-source projects from Connextra-style user stories reverse-engineered from them, with the three prompt exemplars excluded from the primary analysis ($N = 258$). The main contribution is an auditable multi-domain evaluation that separates parser compliance from structural resemblance and demonstrates, through defect auditing and perturbation, why embedding similarity cannot certify business correctness. The two research questions produce different answers. For RQ1, the median skeleton cosine similarity was 0.853 against the reference point of 0.85, and the one-sided exact sign test gave $p = 0.209$ ($\delta = 0.054$; $p = 0.161$ with all 261 files; Wilcoxon $p = 0.474$). We fail to reject H_0 and do not claim that GPT-5.4-mini exceeds the reference point. Only the e-commerce project exceeded it, while the finance project lay significantly below it in both tests and the social-network project only in the Wilcoxon test, and the conclusion would change at a reference point of 0.84. For RQ2, 255 of 258 generated files were valid Gherkin (98.84%, 95% CI [96.64%, 99.76%], $p < 0.001$), so we reject H_0. This corrects the 100% of the submitted version.

The revision analyses limit what the similarity results mean. In an exploratory audit of 60 files at or above the reference point, 25 contained a business-logic

defect, and cosine did not separate defective from non-defective files significantly. Seeded defects left the score almost unchanged. These labels are AI-drafted and human-reviewed, and their agreement with an independent rater was below our protocol's threshold of 0.7 ($\kappa = 0.571$ and 0.609); a consensus round on the 15 doubly labelled files raised the defective files among them from 5 to 8. In a same-time rerun, removing the example lowered the median from 0.852 to 0.811, more than repeated runs of the same prompt vary; a different randomly drawn exemplar per project raised it to 0.873, mostly because of Fineract, but its paired effect stayed within that run-to-run variation, which moved a fifth to a third of the repeated files across 0.85. Parser compliance stayed high under all three prompts (252 to 255 of 255 files).

For practice, and pending a test on independently written stories, the supported use case is assisted drafting: a QA engineer can start from a valid feature file and focus review on expected outcomes, conditions, data values and criterion coverage. The defect evidence behind this advice is exploratory, comes from a high-similarity sample stratified by project, and does not estimate the defect rate of the whole corpus. Neither a parser pass nor a high similarity score supports autonomous acceptance. Future work should (1) use independently written requirements, (2) validate semantics with criterion-level review by independent raters or with execution against step definitions, and (3) compare retrieval-augmented and open-source baselines on the same files.

Acknowledgments. We thank Tran Tuan Khang, Vo Hieu Chuong and Tong Vy Thuan for their contributions to the first version of this study. GPT-5.4-mini is the system under evaluation and was also used to write the user stories from the reference files (Section 3). In the revision, an AI coding assistant (OpenAI Codex) drafted the 60 defect-audit labels, which an author reviewed, and AI assistants (OpenAI Codex and Anthropic Claude) helped write analysis scripts and draft text; Claude also prepared reading aids for the 15-file rater (rubric notes, an acceptance-criteria map, and reading sheets and a glossary for Fineract) that contained no labels. NotebookLM was used to locate passages in the related-work sources; every quoted fact was checked against the source PDFs. All numbers were recomputed from the released data and scripts, and the authors take responsibility for the content.

Data and Ethics. This study involves no human participants; the empirical subjects are public software artifacts and model-generated Gherkin scenarios. Human-subjects review is therefore not applicable. The replication package contains the dataset, prompts, scripts, generated outputs, API logs, audit labels and revision analyses at https://github.com/simonhoang611/RT-SWT-004-Group4.

Disclosure of Interests. The authors declare no external funding and no conflicts of interest.

References

1. Rathnayake, A., Shahin, M., Abaei, G.: Behaviour Driven Development Scenario Generation with Large Language Models. arXiv:2603.04729 (2026). https://doi.org/10.48550/arXiv.2603.04729

2. Karpurapu, S., Myneni, S., Nettur, U., Gajja, L.S., Burke, D., Stiehm, T., Payne, J.: Comprehensive Evaluation and Insights Into the Use of Large Language Models in the Automation of Behavior-Driven Development Acceptance Test Formulation. IEEE Access 12, 58715–58721 (2024). https://doi.org/10.1109/ACCESS.2024.3391815
3. Ferreira, M., Viegas, L., Faria, J.P., Lima, B.: Acceptance Test Generation with Large Language Models: An Industrial Case Study. In: IEEE/ACM International Conference on Automation of Software Test (AST), pp. 1–11 (2025). https://doi.org/10.1109/AST66626.2025.00007
4. Fernandes, H., Perkusich, M., Albuquerque, D., Silva, I., Santos, D.F.S., Gorgônio, K., Perkusich, A.: A Comparative Study of LLMs for Gherkin Generation. In: Brazilian Symposium on Software Engineering (SBES), pp. 171–181 (2025). https://doi.org/10.5753/SBES.2025.9888
5. Fonseca, P.L., Lima, B., Faria, J.P.: Streamlining Acceptance Test Generation for Mobile Applications Through Large Language Models: An Industrial Case Study. In: 40th IEEE/ACM International Conference on Automated Software Engineering (ASE), pp. 3311–3321 (2025). https://doi.org/10.1109/ASE63991.2025.00273
6. Zhang, Q., Shang, Y., Fang, C., Gu, S., Yu, S., Zhou, J., Chen, Z.: TestBench: Evaluating Class-Level Test Case Generation Capability of Large Language Models. Frontiers of Computer Science 21(6), 2106202 (2027), published online 13 October 2025. https://journal.hep.com.cn/fcs/EN/10.1007/s11704-025-50078-9, last accessed 2026/10/28.
7. Chang, H.-F., Shokrolah Shirazi, M.: A Systematic Approach for Assessing Large Language Models' Test Case Generation Capability. Software 4(1), 5 (2025). https://doi.org/10.3390/software4010005
8. Wang, Z., Wang, W., Li, Z., Wang, L., Yi, C., Xu, X., Cao, L., Su, H., Chen, S., Zhou, J.: XUAT-Copilot: Multi-Agent Collaborative System for Automated User Acceptance Testing with Large Language Model. arXiv:2401.02705 (2024). https://doi.org/10.48550/arXiv.2401.02705
9. Liu, J., Xia, C.S., Wang, Y., Zhang, L.: Is Your Code Generated by ChatGPT Really Correct? Rigorous Evaluation of Large Language Models for Code Generation. In: Advances in Neural Information Processing Systems 36 (NeurIPS 2023), pp. 21558–21572 (2023). https://doi.org/10.52202/075280-0943
10. Tong, W., Zhang, T.: CodeJudge: Evaluating Code Generation with Large Language Models. In: Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (EMNLP), pp. 20032–20051 (2024). https://doi.org/10.18653/v1/2024.emnlp-main.1118
11. Kang, S., Yoon, J., Yoo, S.: Large Language Models are Few-shot Testers: Exploring LLM-based General Bug Reproduction. In: 45th IEEE/ACM International Conference on Software Engineering (ICSE), pp. 2312–2323 (2023). https://doi.org/10.1109/ICSE48619.2023.00194
12. Fakhoury, S., Naik, A., Sakkas, G., Chakraborty, S., Lahiri, S.K.: LLM-Based Test-Driven Interactive Code Generation: User Study and Empirical Evaluation. IEEE Transactions on Software Engineering 50(9), 2254–2268 (2024). https://doi.org/10.1109/TSE.2024.3428972
13. Harman, M., Ritchey, J., Harper, I., Sengupta, S., Mao, K., Gulati, A., Foster, C., Robert, H.: Mutation-Guided LLM-based Test Generation at Meta. In: Companion Proceedings of the 33rd ACM International Conference on the Foundations of Software Engineering (FSE Companion), pp. 180–191 (2025). https://doi.org/10.1145/3696630.3728544

14. Zadenoori, M.A., Dąbrowski, J., Alhoshan, W., Zhao, L., Ferrari, A.: Large Language Models (LLMs) for Requirements Engineering (RE): A Systematic Literature Review. arXiv:2509.11446 (2025). https://doi.org/10.48550/arXiv.2509.11446
15. Baltes, S., Angermeir, F., Arora, C., Muñoz Barón, M., Chen, C., Böhme, L., Calefato, F., Ernst, N., Falessi, D., Fitzgerald, B., Fucci, D., He, J., Treude, C., Kalinowski, M., Lambiase, S., Russo, D., Lungu, M., Martinez Montes, C., Prechelt, L., Ralph, P., van Tonder, R., Wagner, S.: Guidelines for Empirical Studies in Software Engineering involving Large Language Models. arXiv:2508.15503v8 (2026). https://doi.org/10.48550/arXiv.2508.15503
16. Reimers, N., Gurevych, I.: Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. In: EMNLP-IJCNLP, pp. 3982–3992 (2019). https://doi.org/10.18653/v1/D19-1410
17. Sylius: Sylius, branch 2.3, commit 9ab28ca. https://github.com/Sylius/Sylius/tree/9ab28ca4b5ad5797216fe14e72e4d6cfe29c3fdb, last accessed 2026/10/28.
18. Apache Software Foundation: Apache Fineract, branch develop, commit 5a93304. https://github.com/apache/fineract/tree/5a93304677061501f5d0c243b8dbf84752e08b0b, last accessed 2026/10/28.
19. diaspora* contributors: diaspora*, branch develop, commit b7bf1dc. https://github.com/diaspora/diaspora/tree/b7bf1dc33566c036a3d788dd8876e97c30905842, last accessed 2026/10/28.
20. OpenAI: GPT-5.4 Mini model documentation. https://developers.openai.com/api/docs/models/gpt-5.4-mini, last accessed 2026/10/28.
21. OpenAI: Create chat completion, API reference. https://developers.openai.com/api/reference/resources/chat/subresources/completions/methods/create, last accessed 2026/10/28.
22. Sentence Transformers: all-MiniLM-L6-v2 model card, revision 1110a24. https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2/tree/1110a243fdf4706b3f48f1d95db1a4f5529b4d41, last accessed 2026/10/28.
23. behave developers: behave v1.3.3 release. https://github.com/behave/behave/releases/tag/v1.3.3, last accessed 2026/10/28.
24. Cohen, J.: A Coefficient of Agreement for Nominal Scales. Educational and Psychological Measurement 20(1), 37–46 (1960). https://doi.org/10.1177/001316446002000104
25. Arcuri, A., Briand, L.C.: A Practical Guide for Using Statistical Tests to Assess Randomized Algorithms in Software Engineering. In: 33rd International Conference on Software Engineering, pp. 1–10 (2011). https://doi.org/10.1145/1985793.1985795
26. Agresti, A.: Categorical Data Analysis, 2nd edn. John Wiley & Sons (2002). https://doi.org/10.1002/0471249688
27. Efron, B., Tibshirani, R.J.: An Introduction to the Bootstrap. Monographs on Statistics and Applied Probability, vol. 57. Chapman & Hall, New York (1993)
28. Clopper, C.J., Pearson, E.S.: The Use of Confidence or Fiducial Limits Illustrated in the Case of the Binomial. Biometrika 26(4), 404–413 (1934). https://doi.org/10.1093/biomet/26.4.404
29. Kruskal, W.H., Wallis, W.A.: Use of Ranks in One-Criterion Variance Analysis. Journal of the American Statistical Association 47(260), 583–621 (1952). https://doi.org/10.1080/01621459.1952.10483441

30. Holm, S.: A Simple Sequentially Rejective Multiple Test Procedure. Scandinavian Journal of Statistics 6(2), 65–70 (1979). https://www.jstor.org/stable/4615733

A Prompt, Skeleton Example and Additional Results

Listing 1 gives the user prompt and Listing 2 the system messages (Section 3), Table 6 maps each project to its exemplars, Table 7 illustrates the skeleton extraction of Section 3.4, Fig. 2 and Fig. 3 show the distribution of skeleton cosine in C0 overall and per project, Table 8 gives the reference-point sensitivity analysis of Section 4.3, and Table 9 gives the parser results of RQ2.

Listing 1. User prompt of conditions C0 and C2, verbatim except for the three braced placeholders; the first line ends with a space. C1 drops the second line and both example blocks.

```
Generate a BDD Gherkin file for the following user story.
Follow the exact style, wording, and conventions shown in the Example.

EXAMPLE USER STORY:
{example_user_story}

EXAMPLE GHERKIN:
{example_gherkin}

---
NOW GENERATE FOR THIS USER STORY:
{user_story}
```

Listing 2. System messages, verbatim. Each project keeps its message in C0, C1 and C2.

```
Sylius: You are an expert QA Engineer for Sylius, an E-commerce platform
    using the Behat framework. You write business-focused Gherkin tests.
    Focus on product variants, shopping carts, and UI interactions using
    business language.

Apache Fineract: You are an expert QA Engineer for Apache Fineract, a
    core banking system. Your signature style is using detailed Data
    Tables in the Background block to mock extensive financial data (e.g.
     LoanProduct, clients). Write highly technical, data-driven Gherkin.

Diaspora: You are an expert QA Engineer for Diaspora, a Ruby on Rails
    social network using Capybara and Cucumber. Your signature style is
    writing concrete UI automation steps using CSS selectors (e.g. '
    within ".class"') and exact UI actions.
```

Table 6. Exemplar per project and condition (dataset id and reference file). The example story is the dataset story of that id and the example Gherkin is its full reference file.

Project	Condition	Id	Reference file
Sylius	C0	2	`being_unable_to_see_cart.feature`
	C2	86	`receiving_discount_based_on_total_of_items_from_specific_taxon.feature`
Fineract	C0	102	`WorkingCapitalNearBreachConfiguration.feature`
	C2	119	`LoanInterestPaymentWaiver.feature`
Diaspora	C0	202	`media-embed.feature`
	C2	203	`connects_users.feature`

The replication package holds the components of every prompt as separate files in the folder `revision/prompts/`: the three system messages of Listing 2, the one-shot template of Listing 1, the zero-shot template of C1, the story and reference Gherkin of each of the six exemplars (subfolder `exemplars/`), and the mapping of Table 6 (`exemplar_mapping.csv`). A C0 or C2 prompt is the one-shot template with the story and Gherkin of the project's exemplar in place of `{example_user_story}` and `{example_gherkin}`. The script `revision/prompts/make_prompt_artifacts.py` rebuilds these files from the dataset and the generation code without calling the API. It checks that the system messages equal those of the original notebook and that the C0 prompts it assembles equal those of the notebook for all 261 stories. All files use LF line breaks, and their SHA-256 hashes are listed below (in `MANIFEST_SHA256.txt` the exemplar files carry the prefix `exemplars/`). C0 was run on 19 July 2026 and C1 and C2 on 16 October 2026, as recorded in the API logs. In those logs the prompt-token difference between C0 or C2 and C1 is the same for every file of a project (C0: 271, 1,910 and 299 tokens for Sylius, Fineract and Diaspora; C2: 960, 9,413 and 965), which is consistent with one fixed exemplar block per project. The version used for this paper is the release tag `fisat2026-camera-ready`, https://github.com/simonhoang611/RT-SWT-004-Group4/tree/fisat2026-camera-ready.

```
exemplar_mapping.csv
  982a690189c23702e9f8ac7b656a9072eab1af252a3b3ba711966c5432222b75
ex_102.feature
  3125864ffcac5dcb98c849b4d138771005ee1ad8405e7154f4ae9463d4388222
ex_102_user_story.txt
  58d9a3af1effd231652d7866f3dccabfc7c5ff4a6cce48816ae963f39558c4d0
ex_119.feature
  bf6699e8e799aa5cf4f850030bc22338c72fd85f85972076a30ba7e538b5dc51
ex_119_user_story.txt
  db4a5a90db39985ae0dba288f1148ce783f661e93da038543d6257a072ab60c0
ex_2.feature
  f24b39313e8729427c4a57f87a829015947e75d3f01f2ec84ec2b7cc477530cc
```

```
ex_202.feature
  1ef601612044212a7497f9cd7783efda22227b245355e848f999b5ca9ee26c7b
ex_202_user_story.txt
  db47c0704403977bb012616803c90ef5b7544bc16fe7983234846912e83a09ae
ex_203.feature
  358d7055e7b9d35685a83ac3b7e833dbd6bbf3d33c9ed06284825dd16b4fc5b5
ex_203_user_story.txt
  06513657140c86fdb09325892bdcb46779e62b5a165c6dc6931d49d1e512b242
ex_2_user_story.txt
  a87eb54d5e418bd8b03d9492ba3d589edc0063ac08b36246bd6ff819410392be
ex_86.feature
  2a8802083c191d1d22f3d9d96185f48602dfe7827945124e4f100f42d998c193
ex_86_user_story.txt
  c20891f56ce4fdb8598f4079a3a94c0dd94a51bc0a8b2be430246afc8313eebd
system_message_ecommerce.txt
  64456030e3229cf63ce9f23a574433e67bc707864dd365e6aca842cab0c82498
system_message_finance.txt
  2193526c6d69c6a09c8f6a34d3a37eafe3ea3f00d0604a47f85b47b7b7c27afd
system_message_social.txt
  24bd9e8f470cc8f5fae0ea4d88b6413d70fd473ed0e4e25dfa75a197c56c19b9
user_prompt_one_shot_template.txt
  e8908eb5441c9e8c8d3f15136d8295d9b011690a0f99b5d3a9d98356829d6387
user_prompt_zero_shot_template.txt
  95645cbaf0b8fae23aa33b68137a0f95d02c626c26db2c91533d097fb6a05364
```

Table 7. Skeleton extraction on a short Sylius-style fragment.

Original line	Skeleton line
`@ui`	(removed)
`Given a product "T-Shirt" priced at "$20.00"`	`Given a product <PARAM> priced at <PARAM>`
`When I add 3 units to the cart`	`When I add <NUM> units to the cart`
`| name | price |`	(removed)
`Then the total should be "$60.00"`	`Then the total should be <PARAM>`

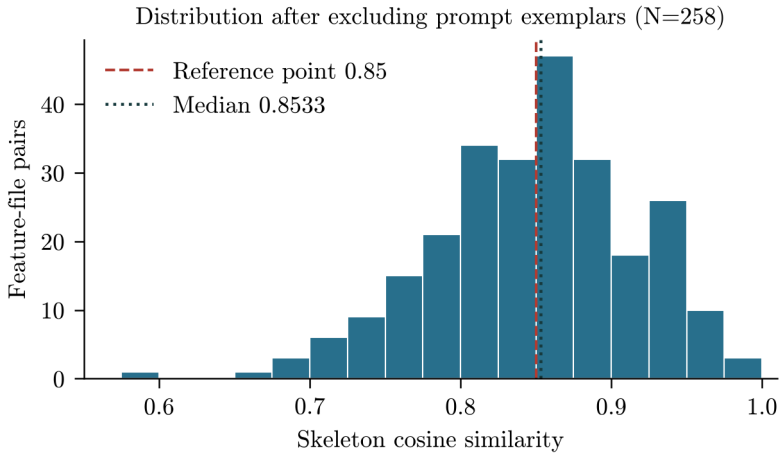


Fig. 2. Distribution of skeleton cosine similarity for the 258 non-exemplar files (condition C0).

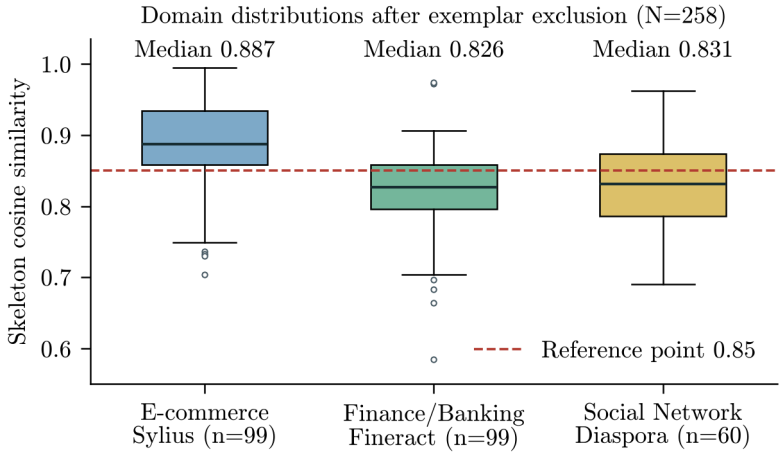


Fig. 3. Skeleton cosine similarity per project (C0, $N = 258$). The dashed line marks 0.85.

Table 8. Reference-point sensitivity (C0, $N = 258$). The p-value is the one-sided Wilcoxon test that the median exceeds t; the exact sign test gives the same decisions at every t ($p = 0.002$ at 0.84 and 0.209 at 0.85).

t	Share $\geq t$ [95% CI]	Median $-t$	p	Sylius	Fineract	Diaspora
0.80	78.3% [72.8, 83.2]	+0.053	< 0.001	91.9%	73.7%	63.3%
0.82	67.8% [61.8, 73.5]	+0.033	< 0.001	87.9%	56.6%	53.3%
0.84	59.3% [53.0, 65.4]	+0.013	0.010	84.8%	40.4%	48.3%
0.85	52.7% [46.4, 58.9]	+0.003	0.474	77.8%	34.3%	41.7%
0.86	43.4% [37.3, 49.7]	−0.007	0.988	73.7%	22.2%	28.3%
0.88	30.6% [25.1, 36.6]	−0.027	1.000	55.6%	12.1%	20.0%
0.90	22.1% [17.2, 27.7]	−0.047	1.000	45.5%	4.0%	13.3%

Table 9. Parser compliance of condition C0 (behave 1.3.3 parser).

Statistic	$N = 258$ (primary)	$N = 261$
Parser PASS / FAIL	255 / 3	258 / 3
ESR	98.84%	98.85%
95% Clopper–Pearson CI	[96.64%, 99.76%]	[96.68%, 99.76%]
Exact binomial p ($p_0 = 0.80$)	4.6×10^{-21}	2.4×10^{-21}

B Perturbation Test

Design. The perturbation test measured whether the metric reacts to seeded defects. From the reference files, 120 scenarios (40 per project, fixed seed) were extracted, and eight operators were applied where applicable: changing a number, a quoted value, a comparison operator or a negation, dropping a Then or a Given step, swapping a When with a Then, and moving a Then to the front. Each mutant was compared with its original scenario using the same skeleton metric.

Results. The 120 extracted scenarios yielded 683 applicable mutants. Of these, 29 mutants do not parse, and the extraction omitted the Background of 74 of the 120 source scenarios. Restricting to the 623 pairs in which both texts parse and the original has no keyword-order flag, every mutant keeps a skeleton cosine of at least 0.8995 with its original. At 0.90, 622 of 623 mutants would still pass, and at 0.95, 97.9%. For changed numbers and quoted values (179 pairs) the skeletons are identical, which follows from masking and is not an empirical finding. Swapped When/Then steps and Then steps moved to the front (210 pairs) score at least 0.954, although the keyword-order check flags all of them. Because the test compares single scenarios and the main metric compares whole files, these results show insensitivity at the scenario level and do not estimate a false-negative rate for whole files.